EXPERIMENT-62(HDOC)

C Programming

Program: Find the Missing Number Using Pointers

Step 1: Understand the Problem Statement

- Input: The size n and the n elements of a variable-length integer array `nums[]`. The elements are distinct integers taken from the range 0 to n, with exactly one number of that range missing.
- Output: The missing number, displayed as an integer in the form `Missing number: x`.
- The array is read in main() and passed to `missing_number(int *nums, int n)` using its address; inside the function the elements are accessed only through a pointer p. The function returns nothing and displays the result itself.
- The range 0 to n contains n + 1 numbers but the array holds only n of them, so exactly one is absent. Because all elements are distinct, no value is counted twice.
- The sum formula is used: the total of 0 + 1 + ... + n is `n*(n+1)/2`. Subtracting every element of the array from this total leaves the missing number.

Rule(s) used: Sum of 0 to n = n(n+1)/2. Missing number = n(n+1)/2 − (sum of all elements of nums[]). For example, n = 3: expected sum = 6, array 3 0 1 sums to 4, so the missing number is 6 − 4 = 2.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Size: 3 Array: 3 0 1	Missing number: 2	Missing number: 2	Pass
2	Size: 2 Array: 0 1	Missing number: 2	Missing number: 2	Pass
3	Size: 9 Array: 9 6 4 2 3 5 7 0 1	Missing number: 8	Missing number: 8	Pass
4	Size: 1 Array: 0	Missing number: 1	Missing number: 1	Pass
5	Size: 4 Array: 1 2 3 4	Missing number: 0	Missing number: 0	Pass

Step 3: Algorithm

1. Start the program.
2. Read the size n, declare the variable-length array `nums[n]` and read its n elements.
3. Call `missing_number(nums, n)`, passing the address of the array.
4. In `missing_number()`, set `missing = n * (n + 1) / 2`, the sum of the numbers from 0 to n.

5. Set pointer `p = nums` and repeat while `p < nums + n`: subtract `*p` from `missing` and advance p.

6. After the loop, `missing` holds the absent number; display `Missing number: missing`.

7. Stop the program.

Evaluation against Step 2 test cases: for n = 3 with array 3 0 1, the expected sum is 6 and the array sums to 4, so 6 − 4 = 2; for n = 2 with array 0 1, 3 − 1 = 2; for n = 9 with array 9 6 4 2 3 5 7 0 1, 45 − 37 = 8; for n = 1 with array 0, 1 − 0 = 1; and for n = 4 with array 1 2 3 4, 10 − 10 = 0 (zero itself can be the missing number) — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-63: Missing Number using pointers and the sum formula.

   The size n and the elements of a variable-length integer array nums[] are
   read in main(). The array holds n DISTINCT integers taken from the range
   0 to n, so exactly one number of that range is missing. The array is
   passed to missing_number() using its address, and the function accesses
   the elements only through a pointer.

   Idea: the sum of all numbers from 0 to n is n*(n+1)/2. Subtracting every
   element of the array from this sum leaves exactly the missing number.
*/

#include <stdio.h>

/* Function prototype */
void missing_number(int *nums, int n);

int main() {
    int n, i;

    printf("Enter size: ");
    scanf("%d", &n);

    int nums[n];                      /* variable-length array */

    printf("Enter %d distinct numbers from 0 to %d: ", n, n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    /* pass the address of the array to the function */
    missing_number(nums, n);

    return 0;
}

/* Finds and displays the one number missing from the range 0 to n */
void missing_number(int *nums, int n) {
    int *p;
    int missing = n * (n + 1) / 2;    /* sum of 0 + 1 + ... + n */

    /* subtract every element of the array using a pointer */
    for (p = nums; p < nums + n; p++)
        missing -= *p;

    printf("Missing number: %d\n", missing);
}
```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`gcc day63.c -o a.out`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

